A REPORT

ON

RESTAURANT MANAGEMENT SYSTEM

by

NAME OF STUDENTS:	REGISTRATION NUMBER:
R. Vidhya	AP24110011915
G. Chaitanya Siri	AP24110011935
M. Jyothika Reddy	AP24110012065
N. Teja Sri	AP24110012101

Prepared in the partial fulfilment of the

Project Based learning of course CSE 209- Database Management System

SRM UNIVERSITY, AP

Abstract:

The Restaurant Management System is a database-driven application designed to streamline and automate the daily operations of a restaurant. This project focuses on efficiently managing various aspects such as customer information, orders, staff, chefs, and servers using fundamental concepts of Database Management Systems (DBMS). The system aims to reduce manual effort, minimize errors, and improve overall operational efficiency by maintaining structured and well-organized data.

In this system, customers can place orders, which are then processed and handled by chefs and served by servers. The database maintains detailed records of all entities involved, including restaurant details, staff members, chefs with their specialties, servers assigned to tables, and customer information. Relationships between these entities are clearly defined using an Entity-Relationship (ER) model, ensuring data consistency and integrity. The ER diagram is further converted into relational tables, which are normalized up to the Third Normal Form (3NF) to eliminate redundancy and dependency issues.

The project also demonstrates the use of Structured Query Language (SQL) for performing various database operations such as data insertion, retrieval, updating, and deletion. Advanced SQL queries including subqueries, aggregate functions, and joins are implemented to extract meaningful insights from the data. Additionally, views are created to simplify complex queries and enhance data security by restricting access to specific information.

Overall, the Restaurant Management System provides a practical implementation of DBMS concepts in a real-world scenario. It ensures efficient data handling, improves service quality, and supports better decision-making within the restaurant. This project highlights the importance of database systems in modern applications and showcases how structured data management can significantly enhance business operations.

Project Background:

In today's fast-paced world, the food service industry has grown rapidly, and restaurants are required to handle a large number of customers, orders, and staff efficiently. Traditionally, many small and medium-sized restaurants rely on manual methods such as handwritten notes, registers, or basic spreadsheets to manage their daily operations. These methods are time-consuming, prone to human errors, and often lead to issues such as misplaced orders, incorrect billing, and poor coordination between staff members. As the volume of data increases, managing and retrieving information becomes even more difficult.

With the advancement of technology, Database Management Systems (DBMS) have become an essential tool for organizing, storing, and managing large amounts of data in a structured manner. A DBMS allows multiple users to access data simultaneously while maintaining accuracy, consistency, and security. It eliminates redundancy, ensures data integrity, and provides efficient ways to retrieve and manipulate data using Structured Query Language (SQL). These features make DBMS highly suitable for applications like restaurant management systems.

The Restaurant Management System project is developed to address the limitations of traditional management methods by providing a computerized solution. This system integrates various components of restaurant operations, including customer management, order processing, staff coordination, and service management. By using an Entity-Relationship (ER) model, the system clearly defines the relationships between entities such as customers, orders, chefs, servers, and the restaurant itself. This structured approach helps in designing an efficient and scalable database.

Overall, the project shows how DBMS can efficiently modernize restaurant operations by improving accuracy, saving time, and enhancing customer experience, while applying database concepts to real-world problems.

Description of the Project:

The **Restaurant Management System** is designed to efficiently store, manage, and organize all data related to restaurant operations in a structured manner. To overcome issues like errors, redundancy, and inefficiency in manual systems, this system uses a relational database approach. Data is divided into multiple tables such as Restaurant, Staff, Chefs, Servers, Customers, and Orders, which are interconnected using primary keys and foreign keys to maintain consistency and eliminate redundancy.

The **Restaurant** table serves as the central entity of the system. It stores essential information such as R_ID (Primary Key), R_Name, R_Contact, and R_Address. It has a one-to-many relationship with the Staff, Chefs, and Servers tables, meaning a single restaurant can have multiple employees. This relationship helps in organizing employee data efficiently.

The **Staff** table stores details of employees including St_ID (Primary Key), St_Name, St_Role, St_Contact, and St_Address. The role attribute helps identify responsibilities of each staff member. It includes R_ID as a foreign key, creating a many-to-one relationship with the Restaurant table.

The **Chefs** table contains information such as Ch_ID (Primary Key), Ch_Name, Ch_Speciality, Ch_Contact, and Ch_Address. It includes R_ID as a foreign key and has a one-to-many relationship with the Orders table through the "Prepare" relationship, indicating that one chef can prepare multiple orders.

The **Servers** table maintains details like Se_ID (Primary Key), Se_Name, Se_Contact, Se_Address, and Se_Table. It includes R_ID as a foreign key and is connected to the Orders table through the "Serves" relationship, where one server can serve multiple orders.

The **Customer** table stores customer details such as C_ID (Primary Key), C_Name, C_Contact, C_Age, and C_Mail. It has a one-to-many relationship with the Orders

table, as one customer can place multiple orders. This helps in maintaining customer records and tracking order history.

The **Orders** table records all order-related information including O_ID (Primary Key), O_Name, and O_DateTime. It contains foreign keys such as C_ID, Ch_ID, and Se_ID, connecting it to Customer, Chefs, and Servers. Each order is associated with one customer, one chef, and one server, ensuring proper coordination and tracking.

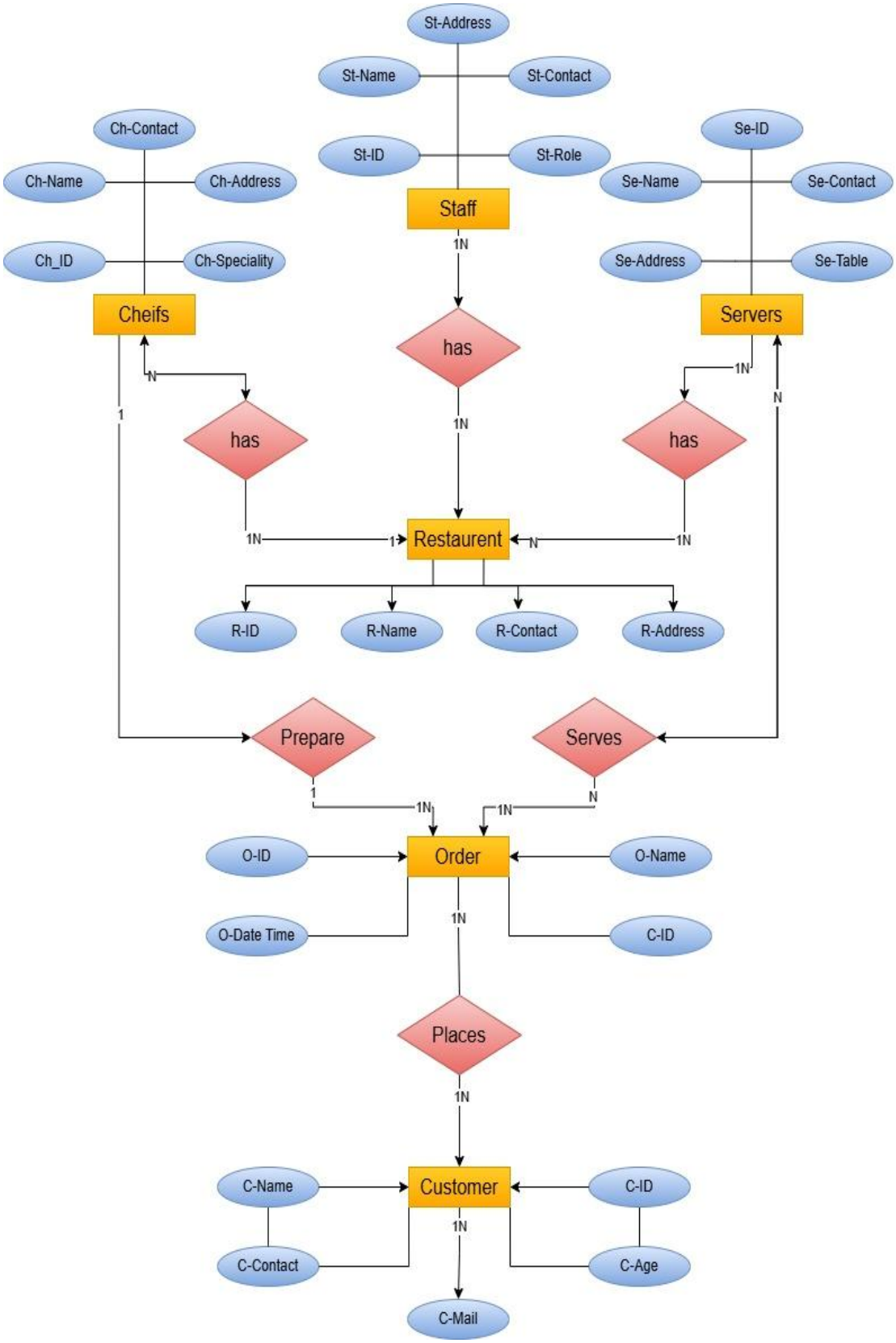
The system uses well-defined relationships like one-to-many and many-to-one to organize data efficiently. Primary keys uniquely identify records, while foreign keys connect related tables. This improves data consistency, reduces redundancy, and ensures smooth restaurant operations.

Additionally, the system enhances coordination between different entities such as customers, staff, chefs, and servers. It enables better communication and workflow management within the restaurant. This leads to improved service quality and faster order processing, ultimately increasing customer satisfaction.

Furthermore, the structured design of the system makes it highly scalable and flexible for future enhancements. New features such as online ordering, payment integration, and inventory management can be easily added. This ensures that the system remains adaptable to changing business needs and technological advancements.

Finally, the system provides better control and monitoring of restaurant activities by maintaining accurate and up-to-date records. It simplifies decision-making for management by providing reliable data and insights. Overall, the Restaurant Management System improves efficiency, productivity, and overall performance of the restaurant.

ER Diagram:


St-Address
St-Name
St-Contact
St-ID
St-Role
Staff
Ch-Contact
Ch-Name
Ch-Address
Ch_ID
Ch-Speciality
Cheifs
Se-ID
Se-Name
Se-Contact
Se-Address
Se-Table
Servers
has
has
has
1N
N
1
1N
N
1N
1
1N
N
Restaurent
R-ID
R-Name
R-Contact
R-Address
Prepare
Serves
1
1N
1N
N
O-ID
Order
O-Name
O-Date Time
C-ID
1N
Places
1N
C-Name
Customer
C-ID
C-Contact
C-Age
1N
C-Mail

Description of ER Diagram:

The given ER diagram illustrates a **Restaurant Management System**, which is designed to manage and organize various activities within a restaurant efficiently. The system consists of several key entities, including **Restaurant, Staff, Chefs, Servers, Orders, and Customers**, each having specific attributes and roles.

The **Restaurant** entity is the central component of the system and contains attributes such as R-ID, R-Name, R-Contact, and R-Address. It is connected to multiple entities through different relationships. The restaurant employs **Staff, Chefs, and Servers**, forming one-to-many relationships, which means a single restaurant can have multiple employees in each category.

The **Staff** entity includes attributes like St-ID, St-Name, St-Role, St-Contact, and St-Address. Similarly, the **Chefs** entity contains Ch-ID, Ch-Name, Ch-Contact, Ch-Address, and Ch-Speciality, representing the skills or specialization of chefs. The **Servers** entity includes Se-ID, Se-Name, Se-Contact, Se-Address, and Se-Table, which indicates the table assigned to each server.

The relationships between these entities define the workflow of the restaurant. Chefs are associated with the **Prepare** relationship, indicating that they are responsible for preparing food orders. Servers are linked through the **Serves** relationship, meaning they deliver the prepared food to customers.

The **Order** entity plays a crucial role in connecting different parts of the system. It includes attributes such as O-ID, O-Name, and O-Date Time. Orders are prepared by chefs and served by servers, forming many-to-one or one-to-many relationships depending on the interaction.

The **Customer** entity includes attributes like C-ID, C-Name, C-Contact, C-Mail, and C-Age. Customers interact with the system through the **Places** relationship, where a customer can place multiple orders, indicating a one-to-many relationship.

Overall, the ER diagram clearly organizes the restaurant system into entities and relationships, reducing redundancy and maintaining data consistency.

Conversion of ER diagram into Tables:

Restaurant Table:

Code:

```
CREATE TABLE Restaurant (
    R_ID INT PRIMARY KEY,
    R_Name VARCHAR(50),
    R_Contact VARCHAR(15),
    R_Address VARCHAR(100)
);
DESC Restaurant;
```

Output:

Field	Type	Null	Key	Default	Extra
R_ID	int	NO	PRI	NULL	
R_Name	varchar(50)	YES		NULL	
R_Contact	varchar(15)	YES		NULL	
R_Address	varchar(100)	YES		NULL	

Staff Table:

Code:

```
CREATE TABLE Staff (
    St_ID INT PRIMARY KEY,
    St_Name VARCHAR(50),
    St_Role VARCHAR(30),
    St_Contact VARCHAR(15),
    St_Address VARCHAR(100),
    R_ID INT,
    FOREIGN KEY (R_ID) REFERENCES Restaurant(R_ID)
);
DESC Staff;
```

Output:

Field	Type	Null	Key	Default	Extra
St_ID	int	NO	PRI	NULL	
St_Name	varchar(50)	YES		NULL	
St_Role	varchar(30)	YES		NULL	
St_Contact	varchar(15)	YES		NULL	
St_Address	varchar(100)	YES		NULL	
R_ID	int	YES	MUL	NULL	

Chefs Table:

Code:

```
CREATE TABLE Chefs (
    Ch_ID INT PRIMARY KEY,
    Ch_Name VARCHAR(50),
    Ch_Speciality VARCHAR(50),
    Ch_Contact VARCHAR(15),
    Ch_Address VARCHAR(100),
    R_ID INT,
    FOREIGN KEY (R_ID) REFERENCES Restaurant(R_ID)
);
DESC Chefs;
```

Output:

Field	Type	Null	Key	Default	Extra
Ch_ID	int	NO	PRI	NULL	
Ch_Name	varchar(50)	YES		NULL	
Ch_Speciality	varchar(50)	YES		NULL	
Ch_Contact	varchar(15)	YES		NULL	
Ch_Address	varchar(100)	YES		NULL	
R_ID	int	YES	MUL	NULL	

Servers Table:

Code:

```
CREATE TABLE Servers (
    Se_ID INT PRIMARY KEY,
    Se_Name VARCHAR(50),
```

```
    Se_Contact VARCHAR(15),
    Se_Address VARCHAR(100),
    Se_Table INT,
    R_ID INT,
    FOREIGN KEY (R_ID) REFERENCES Restaurant(R_ID)
);
DESC Servers;
```

Output:

Field	Type	Null	Key	Default	Extra
Se_ID	int	NO	PRI	NULL	
Se_Name	varchar(50)	YES		NULL	
Se_Contact	varchar(15)	YES		NULL	
Se_Address	varchar(100)	YES		NULL	
Se_Table	int	YES		NULL	
R_ID	int	YES	MUL	NULL	

Customer Table:

Code:

```
CREATE TABLE Customer (
    C_ID INT PRIMARY KEY,
    C_Name VARCHAR(50),
    C_Contact VARCHAR(15),
    C_Age INT,
    C_Mail VARCHAR(50)
);
DESC Customer;
```

Output:

Field	Type	Null	Key	Default	Extra
C_ID	int	NO	PRI	NULL	
C_Name	varchar(50)	YES		NULL	
C_Contact	varchar(15)	YES		NULL	
C_Age	int	YES		NULL	
C_Mail	varchar(50)	YES		NULL	

Orders Table:

Code:

```
CREATE TABLE Orders (
    O_ID INT PRIMARY KEY,
    O_Name VARCHAR(50),
    O_DateTime DATETIME,
    C_ID INT,
    Ch_ID INT,
    Se_ID INT,
    FOREIGN KEY (C_ID) REFERENCES Customer(C_ID),
    FOREIGN KEY (Ch_ID) REFERENCES Chefs(Ch_ID),
    FOREIGN KEY (Se_ID) REFERENCES Servers(Se_ID)
);
DESC Orders;
```

Output:

Field	Type	Null	Key	Default	Extra
O_ID	int	NO	PRI	NULL	
O_Name	varchar(50)	YES		NULL	
O_DateTime	datetime	YES		NULL	
C_ID	int	YES	MUL	NULL	
Ch_ID	int	YES	MUL	NULL	
Se_ID	int	YES	MUL	NULL	

Description of Tables:

The database consists of multiple tables, each representing a specific entity in the Restaurant Management System. These tables are designed to store and manage data efficiently using primary keys and foreign keys.

The **Restaurant** table stores basic information about the restaurant such as R_ID, R_Name, R_Contact, and R_Address. The R_ID acts as the primary key and uniquely identifies each restaurant.

The **Staff** table contains details of employees including St_ID, St_Name, St_Role, St_Contact, and St_Address. It includes R_ID as a foreign key, which connects staff members to the restaurant they belong to.

The **Chefs** table stores information about chefs such as Ch_ID, Ch_Name, Ch_Speciality, Ch_Contact, and Ch_Address. The R_ID is used as a foreign key to link chefs with the restaurant.

The **Servers** table maintains details of servers including Se_ID, Se_Name, Se_Contact, Se_Address, and Se_Table. It also contains R_ID as a foreign key to associate servers with a restaurant.

The **Customer** table stores customer details such as C_ID, C_Name, C_Contact, C_Age, and C_Mail. The C_ID acts as the primary key for identifying customers uniquely.

The **Orders** table records all order-related information including O_ID, O_Name, and O_DateTime. It includes foreign keys such as C_ID, Ch_ID, and Se_ID to connect orders with customers, chefs, and servers respectively.

Overall, these tables are interconnected to ensure proper data organization, reduce redundancy, and maintain data consistency within the system.

Normalization of Tables up to 3NF:

Normalization is a systematic process used in database design to organize data efficiently by reducing redundancy and improving data integrity. It involves dividing large tables into smaller, well-structured tables and establishing relationships between them using keys. In the Restaurant Management System, normalization ensures that data is stored logically and consistently.

First Normal Form (1NF)

A table is said to be in First Normal Form if all its attributes contain atomic values and there are no repeating groups or multi-valued attributes. In this system, each table such as Restaurant, Staff, Chefs, Servers, Customer, and Orders contains only single-valued attributes. For example, contact numbers and names are stored as individual fields. Hence, all tables satisfy the conditions of 1NF.

Second Normal Form (2NF)

A table is in Second Normal Form if it is already in 1NF and all non-key attributes are fully functionally dependent on the primary key. In the given system, each table has a single primary key (e.g., R_ID, St_ID, C_ID), and all other attributes depend entirely on that key. There are no partial dependencies, which means the tables meet the requirements of 2NF.

Third Normal Form (3NF)

A table is in Third Normal Form if it is in 2NF and there are no transitive dependencies, meaning non-key attributes do not depend on other non-key attributes. In this system, attributes such as name, contact, and address depend only on their respective primary keys and not on any other non-key attributes. Therefore, all tables are in 3NF.

Conclusion

The tables in the Restaurant Management System are normalized up to Third Normal Form (3NF), reducing redundancy and ensuring data consistency. This improves overall efficiency and makes the system scalable and easy to maintain.

Creation of Data in the Tables:

Restaurant Table:

INSERT INTO Restaurant VALUES

(1, 'Spice Hub', '9876543210', 'Hyderabad'),

(2, 'Food Palace', '9123456780', 'Vijayawada'),

(3, 'Tasty Bites', '9012345678', 'Guntur'),

(4, 'Food Corner', '9988776655', 'Chennai');

SELECT * FROM Restaurant;

R_ID	R_Name	R_Contact	R_Address
1	Spice Hub	9876543210	Hyderabad
2	Food Palace	9123456780	Vijayawada
3	Tasty Bites	9012345678	Guntur
4	Food Corner	9988776655	Chennai
NULL	NULL	NULL	NULL

Staff Table:

INSERT INTO Staff VALUES

(101, 'Ramesh', 'Manager', '9000000001', 'Hyderabad', 1),

(102, 'Sita', 'Cleaner', '9000000002', 'Hyderabad', 1),

(103, 'Arun', 'Assistant', '9000000003', 'Guntur', 3),

(104, 'Kiran', 'Cashier', '9000000004', 'Chennai', 4);

SELECT * FROM Staff;

St_ID	St_Name	St_Role	St_Contact	St_Address	R_ID
101	Ramesh	Manager	9000000001	Hyderabad	1
102	Sita	Cleaner	9000000002	Hyderabad	1
103	Arun	Assistant	9000000003	Guntur	3
104	Kiran	Cashier	9000000004	Chennai	4
NULL	NULL	NULL	NULL	NULL	NULL

Chefs Table:

INSERT INTO Chefs VALUES

(201, 'Kumar', 'South Indian', '9000000011', 'Hyderabad', 1),

(202, 'Raj', 'Chinese', '9000000012', 'Vijayawada', 2),

(203, 'Manoj', 'North Indian', '9000000013', 'Guntur', 3),

(204, 'Ravi', 'Fast Food', '9000000014', 'Chennai', 4);

SELECT * FROM Chefs;

Ch_ID	Ch_Name	Ch_Speciality	Ch_Contact	Ch_Address	R_ID
201	Kumar	South Indian	9000000011	Hyderabad	1
202	Raj	Chinese	9000000012	Vijayawada	2
203	Manoj	North Indian	9000000013	Guntur	3
204	Ravi	Fast Food	9000000014	Chennai	4
NULL	NULL	NULL	NULL	NULL	NULL

Servers Table:

INSERT INTO Servers VALUES

(301, 'Ravi', '9000000021', 'Hyderabad', 5, 1),

(302, 'Anil', '9000000022', 'Vijayawada', 3, 2),

(303, 'Vijay', '9000000023', 'Guntur', 2, 3),

(304, 'Suresh', '9000000024', 'Chennai', 4, 4);

SELECT * FROM Servers;

Se_ID	Se_Name	Se_Contact	Se_Address	Se_Table	R_ID
301	Ravi	9000000021	Hyderabad	5	1
302	Anil	9000000022	Vijayawada	3	2
303	Vijay	9000000023	Guntur	2	3
304	Suresh	9000000024	Chennai	4	4
NULL	NULL	NULL	NULL	NULL	NULL

Customer Table:

INSERT INTO Customer VALUES

(401, 'Priya', '9000000031', 22, 'priya@gmail.com'),

(402, 'Arjun', '9000000032', 25, 'arjun@gmail.com'),

(403, 'Sneha', '9000000033', 21, 'sneha@gmail.com'),

(404, 'Rahul', '9000000034', 28, 'rahul@gmail.com');

SELECT * FROM Customer;

C_ID	C_Name	C_Contact	C_Age	C_Mail
401	Priya	9000000031	22	priya@gmail.com
402	Arjun	9000000032	25	arjun@gmail.com
403	Sneha	9000000033	21	sneha@gmail.com
404	Rahul	9000000034	28	rahul@gmail.com
NULL	NULL	NULL	NULL	NULL

Orders Table:

INSERT INTO Orders VALUES

(501, 'Biryani', '2026-04-15 12:30:00', 401, 201, 301),

(502, 'Noodles', '2026-04-15 13:00:00', 402, 202, 302),

(503, 'Fried Rice', '2026-04-15 13:30:00', 403, 203, 303),

(504, 'Pizza', '2026-04-15 14:00:00', 404, 204, 304);

SELECT * FROM Orders;

O_ID	O_Name	O_DateTime	C_ID	Ch_ID	Se_ID
501	Biryani	2026-04-15 12:30:00	401	201	301
502	Noodles	2026-04-15 13:00:00	402	202	302
503	Fried Rice	2026-04-15 13:30:00	403	203	303
504	Pizza	2026-04-15 14:00:00	404	204	304
NULL	NULL	NULL	NULL	NULL	NULL

SQL queries:

Subqueries

- SELECT C_Name FROM Customer
  WHERE C_ID IN (SELECT C_ID FROM Orders);

C_Name
Priya
Arjun
Sneha
Rahul

- SELECT Ch_Name FROM Chefs
  WHERE Ch_ID IN (SELECT Ch_ID FROM Orders);

Ch_Name
Kumar
Raj
Manoj
Ravi

- SELECT C_Name FROM Customer
  WHERE C_Age > (SELECT AVG(C_Age) FROM Customer);

C_Name
Arjun
Rahul

- SELECT O_Name FROM Orders
  WHERE Se_ID = (SELECT Se_ID FROM Servers WHERE Se_ID = 301);

O_Name
Biryani

- SELECT R_Name FROM Restaurant
  WHERE R_ID IN (SELECT R_ID FROM Staff);

R_Name
Spice Hub
Tasty Bites
Food Corner

Aggregate Functions

- SELECT COUNT(*) AS Total_Customers FROM Customer;

Total_Customers
4

- SELECT AVG(C_Age) AS Avg_Age FROM Customer;

Avg_Age
24.0000

- SELECT MAX(C_Age) AS Max_Age FROM Customer;

Max_Age
28

- SELECT MIN(C_Age) AS Min_Age FROM Customer;

Min_Age
21

- SELECT COUNT(*) AS Total_Orders FROM Orders;

Total_Orders
4

Joins

- SELECT C.C_Name, O.O_Name
  FROM Customer C
  JOIN Orders O ON C.C_ID = O.C_ID;

C_Name	O_Name
Priya	Biryani
Arjun	Noodles
Sneha	Fried Rice
Rahul	Pizza

- SELECT Ch.Ch_Name, O.O_Name
  FROM Chefs Ch
  JOIN Orders O ON Ch.Ch_ID = O.Ch_ID;

Ch_Name	O_Name
Kumar	Biryani
Raj	Noodles
Manoj	Fried Rice
Ravi	Pizza

- SELECT S.Se_Name, O.O_Name
  FROM Servers S
  JOIN Orders O ON S.Se_ID = O.Se_ID;

Se_Name	O_Name
Ravi	Biryani
Anil	Noodles
Vijay	Fried Rice
Suresh	Pizza

- SELECT St.St_Name, R.R_Name
  FROM Staff St
  JOIN Restaurant R ON St.R_ID = R.R_ID;

St_Name	R_Name
Ramesh	Spice Hub
Sita	Spice Hub
Arun	Tasty Bites
Kiran	Food Corner

- SELECT C.C_Name, Ch.Ch_Name, S.Se_Name, O.O_Name
  FROM Orders O
  JOIN Customer C ON O.C_ID = C.C_ID
  JOIN Chefs Ch ON O.Ch_ID = Ch.Ch_ID
  JOIN Servers S ON O.Se_ID = S.Se_ID;

C_Name	Ch_Name	Se_Name	O_Name
Priya	Kumar	Ravi	Biryani
Arjun	Raj	Anil	Noodles
Sneha	Manoj	Vijay	Fried Rice
Rahul	Ravi	Suresh	Pizza

Creation of views using tables:

Customer_Orders View

- CREATE VIEW Customer_Orders AS
  SELECT C.C_Name, O.O_Name, O.O_DateTime
  FROM Customer C
  JOIN Orders O ON C.C_ID = O.C_ID;
  SELECT * FROM Customer_Orders;

C_Name	O_Name	O_DateTime
Priya	Biryani	2026-04-15 12:30:00
Arjun	Noodles	2026-04-15 13:00:00
Sneha	Fried Rice	2026-04-15 13:30:00
Rahul	Pizza	2026-04-15 14:00:00

Chef_Orders View:

- CREATE VIEW Chef_Orders AS
  SELECT Ch.Ch_Name, O.O_Name
  FROM Chefs Ch
  JOIN Orders O ON Ch.Ch_ID = O.Ch_ID;
  SELECT * FROM Chef_Orders;

Ch_Name	O_Name
Kumar	Biryani
Raj	Noodles
Manoj	Fried Rice
Ravi	Pizza

Server_Details View:

- CREATE VIEW Server_Details AS
  SELECT Se.Se_Name, Se.Se_Table, R.R_Name
  FROM Servers Se
  JOIN Restaurant R ON Se.R_ID = R.R_ID;

SELECT * FROM Server_Details;

Se_Name	Se_Table	R_Name
Ravi	5	Spice Hub
Anil	3	Food Palace
Vijay	2	Tasty Bites
Suresh	4	Food Corner

Staff_Restaurant View:

- CREATE VIEW Staff_Restaurant AS

  SELECT St.St_Name, St.St_Role, R.R_Name

  FROM Staff St

  JOIN Restaurant R ON St.R_ID = R.R_ID;

  SELECT * FROM Staff_Restaurant;

St_Name	St_Role	R_Name
Ramesh	Manager	Spice Hub
Sita	Cleaner	Spice Hub
Arun	Assistant	Tasty Bites
Kiran	Cashier	Food Corner

Order_Details View:

- CREATE VIEW Order_Details AS

  SELECT C.C_Name, Ch.Ch_Name, S.Se_Name, O.O_Name, O.O_DateTime

  FROM Orders O

  JOIN Customer C ON O.C_ID = C.C_ID

  JOIN Chefs Ch ON O.Ch_ID = Ch.Ch_ID

  JOIN Servers S ON O.Se_ID = S.Se_ID;

  SELECT * FROM Order_Details;

C_Name	Ch_Name	Se_Name	O_Name	O_DateTime
Priya	Kumar	Ravi	Biryani	2026-04-15 12:30:00
Arjun	Raj	Anil	Noodles	2026-04-15 13:00:00
Sneha	Manoj	Vijay	Fried Rice	2026-04-15 13:30:00
Rahul	Ravi	Suresh	Pizza	2026-04-15 14:00:00